Diffusal: Error Dynamics in Ternary Diffusion Language Models

Abstract

The end goal of Diffusal is a **1.58-bit (ternary) text diffusion model in the Gemma family**: a masked/discrete diffusion language model whose weights are `{-1, 0, +1}`, trained natively at that precision as BitNet b1.58 prescribes. The concrete build target is an A2D (autoregressive-to-diffusion) conversion of **Gemma 4 E2B** with ternary QAT distilled from its FP16 parent (§5.1) — *not* post-training quantization of DiffusionGemma-26B, which this document's own premises rule out. Everything else in this document is the de-risking program for that build.

The core scientific risk is feedback through refinement. Diffusion language models (dLLMs) repeatedly refine a token canvas instead of generating strictly left to right, so the same quantized denoiser is applied to (partially) its own output many times. Diffusal asks: across refinement steps, does quantization error contract or expand? The realistic answer is a **crossover** — contractive at high noise levels, where denoising has strong pull toward the data manifold, and expansive at low noise levels, where the remaining signal is small and relative quantization error dominates. The deliverable is the crossover point, expressed in **mask fraction / noise level** (not step index, which is a sampler hyperparameter and does not transfer across schedules), and reported as a **distribution across samples**, not a single averaged curve — averaging can manufacture a smooth crossover that no individual trajectory has.

The novelty claim is specific. Timestep-aware quantization work in *continuous* image diffusion (PTQ4DM, Q-Diffusion, TDQ) already established that quantization error accumulates across denoising steps, varies by timestep, and that step-dependent precision is the mitigation. What is different in masked *text* diffusion is that **argmax commitment is itself a quantizer**: sub-threshold logit perturbations are discretized away entirely at commit time — a genuine absorption mechanism with no analog in continuous diffusion — while super-threshold perturbations flip a token discretely and irreversibly. This predicts error dynamics that are step-function-like rather than smooth, and it makes the commit/remask policy, not the denoiser alone, the load-bearing component.

Two design commitments follow. First, because commit-and-freeze samplers never revisit committed tokens, the feedback loop under study barely exists for them; Diffusal therefore targets a **sampler with genuine remasking**. Note that LLaDA's "low-confidence remasking" does *not* qualify — its sampler freezes committed tokens permanently and only chooses which still-masked positions to unmask (the "failure to remask" property). True remasking is retrofitted at inference time via **ReMDM**, and every trajectory claim is scoped to the sampler family it was measured on. Second, because ternary PTQ fails on *any* architecture — that is BitNet's own premise — no dLLM quantization result means anything without a **matched autoregressive control at the same bit-width**; only the *excess* degradation of the dLLM over the AR control is evidence about diffusion.

Deployment context still matters — a ternary dLLM that loses on quality-per-VRAM to a smaller INT4 AR model is not a useful artifact — but at the scales this program can afford, dLLMs lose to AR models regardless of quantization, so the AR frontier is **context, not the kill gate**. The kill gate is internal: is the ternary-vs-FP16 gap for the dLLM materially worse than the same gap for a matched AR model?

1. Background, Prior Art, and Scope

Masked and discrete diffusion language models such as MDLM and LLaDA make iterative text denoising a real compression target. BitNet b1.58 argues that ternary LLMs can be competitive when trained for ternary weights from the beginning. These facts do not imply that a ternary diffusion model works, and the interaction between the two is what Diffusal studies on the way to building one.

What is already known. In continuous image diffusion, the question "does quantization error accumulate over denoising steps?" is answered: yes, non-uniformly by timestep, and the standard mitigations are timestep-aware calibration and mixed precision on sensitive steps (PTQ4DM, Q-Diffusion, TDQ). Diffusal must not re-derive this. Its contribution is confined to what is different in the discrete/masked text setting:

1. **Argmax commitment quantizes error.** A logit perturbation smaller than the gap to the runner-up token vanishes at commit time; a larger one flips a token discretely. Continuous diffusion has no such absorption threshold. This is the strongest a-priori reason to think text diffusion could be *more* ternary-tolerant than image diffusion per step — and *less* tolerant per mistake, because a flipped commit is irreversible under most samplers.
2. **The sampler gates the feedback loop.** With commit-and-freeze unmasking, committed tokens are never revisited and the process is closer to any-order autoregression; the "iterated canvas" premise applies only to the still-masked positions' distributions. With remasking samplers, committed tokens can re-enter play. The contract/expand question is therefore **sampler-dependent**, and under commit-and-freeze it largely collapses to "does quantization change the order and identity of commits?" — the §6.1 question. A caution from code inspection: the well-known samplers of LLaDA and Dream are commit-and-freeze despite the "remasking" naming — LLaDA's `low_confidence` mode recomputes the mask set as `(x == mask_id)` and assigns `-inf` confidence to unmasked positions, so a committed token can never re-enter play. Genuine remasking exists as an inference-time retrofit: **ReMDM** (Wang et al.) re-opens committed tokens on any pretrained masked diffusion checkpoint without retraining, with several schedules (cap, rescale, conf, loop). Diffusal's primary sampler is ReMDM on the chosen base model, where the feedback loop genuinely exists; commit-and-freeze is measured as a secondary condition on the same checkpoint — a clean two-condition design that isolates the feedback loop with weights held fixed.

Scope. Diffusal is a measurement-then-build program: Experiments 0–1 establish whether and where ternary error dynamics are dangerous; Experiment 2 is the actual ternary training line that the end-goal model grows out of.

2. Main Claims

1. **PTQ is a baseline and a control, never the path.** BitNet's lesson is that native low-bit training matters; ternary PTQ fails on every architecture, so ternary-PTQ failure on a dLLM is uninformative *unless* it exceeds the failure of a matched AR model quantized identically.
2. **The primary observable is the error trajectory and its crossover, in mask-fraction terms.** Final task score is insufficient; the experiment must show how quantization error evolves as a function of noise level and report the crossover distribution across samples — measured on well-posed protocols (§4), not a single confounded diff.
3. **Contractivity and per-step distortion are different measurements.** Teacher-forced comparison measures per-step distortion at a given state; whether the dynamics contracts an injected error is a separate, perturbation-based measurement (§4 Protocol C). Neither substitutes for the other.
4. **All-ternary behavior matters more than one-module sensitivity.** Per-module ablations are diagnostic, but quantization effects are non-additive.
5. **The go/no-go metric is the gap-of-gaps.** Ternary-vs-FP16 quality gap for the dLLM, compared against the same gap for a matched AR model at the same scale and recipe. The absolute AR frontier at a VRAM budget is reported as context.
6. **A 6 GB 25B deployment is probably dead on arrival.** Weight compression alone is the wrong lever if activations, embeddings, logits, and runtime buffers dominate.

3. Experiment 0: PTQ Failure, With an AR Control

Objective: establish the PTQ degradation profile of the chosen small dLLM *relative to a matched autoregressive control*, at each precision, before building anything on top.

3.1 Model and sampler selection

The platform is chosen to make the matched-control and remasking requirements free rather than expensive:

- **Primary dLLM: MDLM-owt** (`kuleshov-group/mdlm-owt`, ~130M non-embedding parameters, GPT-2 tokenizer, 1M steps / 33B tokens on OpenWebText). Small enough to run every protocol and ablation cheaply, and it is the checkpoint the ReMDM code was built against.
- **Remasking condition: ReMDM sampling** on that checkpoint (inference-time, no retraining; cap/rescale/conf/loop schedules). **Commit-and-freeze condition:** the vanilla MDLM confidence sampler on the same checkpoint. Same weights, two feedback regimes.
- **Matched AR control: the MDLM repo's own AR baseline** (`ar.ckpt`), trained by the same authors on the same OpenWebText data at the same scale in the same codebase — the §5 gap-of-gaps control without training anything. (A SEDD baseline from the same release is available as a second diffusion parameterization if needed. Tokenizer identity is verified: both models are evaluated with the repo's `data=openwebtext-split` config, which sets `tokenizer_name_or_path: gpt2`, at the same `model.length=1024` — MDLM runs as `model=small, backbone=dit, parameterization=subs`, the AR control as `model=small-ar, backbone=ar, parameterization=ar`.)
- **Second scale point: Tiny-A2D 0.5B/0.6B** from the dLLM framework (ZHZisZZ/dllm) — diffusion models converted from AR parents (Qwen/LLaMA/GPT-2) with open recipes. The

conversion recipe means each checkpoint has a *literal AR parent*, an unusually tight control; the framework also provides unified train/eval/sampler infrastructure for the QAT line in §5.

- **Primary context model: DiffusionGemma-26B-A4B** (`google/diffusiongemma-26B-A4B-it`, Apache 2.0) — Google's open-weight block-diffusion MoE in the target model family, with ~4B active parameters, so the Experiment 0 PTQ ladder (INT8/INT4/ternary) runs on it with single-GPU inference (~13 GB weights at INT4). It is measurement-only: 26B-scale QAT is out of budget, it is MoE (routers must stay high precision per §6.3), and it is nearly literally §8′s cautionary 25B model. Its value is testing whether the small-model error-dynamics findings transfer to a real block-diffusion MoE in the family the end-goal model belongs to.
- **Secondary context models: LLaDA-8B, Dream-7B.** Too large for the ablation grid, and their native samplers are commit-and-freeze (§1); results on them are reported as external validity checks, not primary evidence.
- **Sampler validation:** sampler implementations are checked against the sampler-correctness evaluations of Tang et al. (dllm_sampler) so that a buggy sampler is not the confound.

Method:

1. take MDLM-owt as the small dLLM, under both ReMDM and commit-and-freeze sampling (§3.1),
2. take the same release's AR baseline as the matched control,
3. run FP16 baselines for both,
4. apply naive PTQ to both at **INT8, INT4, and ternary** — the precision must be pinned per result; INT8/INT4 findings do not transfer to ternary,
5. apply a timestep/mask-aware PTQ variant inspired by DLLMQuant to the dLLM,
6. compare degradation *deltas*: (dLLM quantized − dLLM FP16) versus (AR quantized − AR FP16) at each precision.

Gate and its meaning. The informative quantity is the excess degradation of the dLLM over the AR control, not dLLM failure in isolation:

- If the dLLM's degradation materially exceeds the AR control's at matched precision, the refinement-dynamics worry is live and Experiments 1–2 are motivated.
- If the two degrade comparably, refinement is not adding measurable fragility at that precision — the thesis re-scopes toward "why is iterated denoising this robust?" and the argmax-absorption mechanism (§1) becomes the object of study.
- Ternary PTQ failing on *both* models is the expected, uninformative outcome; it neither motivates nor kills anything. The INT4⟶ternary trend of the *delta* is the early signal.

4. Experiment 1: Measure Error-Trajectory Dynamics

Objective: characterize how quantization error evolves under iterative refinement — the shape of the per-step distortion curve as a function of mask fraction, whether the dynamics contracts injected errors, and where (if anywhere) absorption turns into amplification.

4.1 The alignment problem

The naive plan — run FP16 and quantized denoisers from the same corrupted canvas and diff their intermediate states — is confounded. As soon as the quantized model commits or remasks a *different* token at step k, the two models are no longer denoising the same canvas at step $k+1$. Any divergence measured after that fork conflates per-step distortion with the trajectories having forked into different problems. Measured naively, "KL grew" can just mean "one early commit disagreement snowballed" — a commit-decision problem (§6.1), not a weight-precision problem.

We therefore run three protocols and keep them separate.

Protocol A — teacher-forced (per-step distortion at FP16 states). At every step, feed the quantized model the exact committed/unmasked canvas the FP16 model produced, so both denoise the identical state. This isolates the per-step distortion ternary weights inject at a fixed state.

Two limits of Protocol A must be stated up front. First, it is **off-policy**: the states are drawn from the FP16 trajectory distribution, which the quantized model might never visit on its own, so its per-step KL characterizes distortion on FP16′s manifold, not the quantized model's. Second, and more important: because every step restarts from the FP16 canvas, **nothing can accumulate — Protocol A cannot observe amplification by construction**. If Protocol A error grows with mask fraction, that is *state-dependent sensitivity* (low-noise states are harder to denoise faithfully at ternary precision), which is useful — it sizes the noise injected at each stage — but it is not evidence of feedback amplification. Earlier drafts conflated these; they are different claims. A mechanical detail: when the quantized model would commit a different *number* of tokens than FP16 at a step, we force FP16′s commit pattern and flag the step; the aggregate flag rate bounds how off-policy the protocol is.

Protocol B — free-running (realistic). Both models run independently from the same start. This is the deployment condition, but its divergence is a mixture of per-step distortion and trajectory forking, so it cannot by itself prove expansion.

Protocol C — perturbation-injection contractivity (the dynamics measurement). Amplification is a property of the dynamics, so measure it on the dynamics directly: inject a controlled perturbation into the **FP16 model's own state** at noise level σ — flip the k least-confident committed tokens, or add calibrated logit noise before commit — and track whether the FP16 sampler contracts or grows that perturbation over subsequent steps (token-Hamming distance to the unperturbed run, as a function of steps-since-injection and of σ). This is a Lyapunov-style measurement and it is independent of quantization. Repeat with the perturbation magnitude set to the Protocol A distortion measured at that σ. Contractive dynamics + bounded per-step distortion $\Rightarrow$ absorption; expansive dynamics at the σ where Protocol A distortion is large $\Rightarrow$ amplification, with the mechanism localized.

The decomposition across protocols is the result. If Protocol C says the dynamics contracts at high mask fraction and expands at low mask fraction, and Protocol A says distortion is largest at low mask fraction, the crossover and its mitigation cost are determined. If free-running (B) diverges

while A stays bounded and C stays contractive, the problem is **commit decisions / schedule**, and the fix is precision on the remasking/confidence path (§6.1), not on the whole network.

4.2 Metrics

All trajectory metrics are reported as a function of **mask fraction / noise level**, not raw step index, and as **per-sample distributions**, not only means.

Primary:

- committed/unmasked token agreement vs FP16 (Protocols A, B) and vs the unperturbed run (Protocol C),
- Hamming/edit distance decay curve after injection (Protocol C) — the contractivity estimate,
- oscillation rate: tokens that repeatedly flip across steps (defined only for the remasking sampler; reported as n/a for commit-and-freeze runs).

Quality anchor (required alongside every divergence metric): divergence measures fidelity to FP16, not quality — generation is multimodal and a diverged completion can be equally good. So every condition also reports perplexity under a fixed larger oracle model and final task score. **A divergence finding with no quality drop is a fidelity result, not a failure**, and is labeled as such.

Supporting:

- KL divergence between FP16 and quantized token distributions by noise level — supporting rather than primary because over a large vocab it is tail-dominated and temperature-sensitive, and near the mask token the distributions are near-degenerate,
- confidence-calibration drift by noise level.

Thresholds for "grows," "bounded," and "contracts" are **pre-registered** in the experiment configs before runs, not chosen after seeing curves; the kill criteria in §10 reference those declared numbers.

4.3 What we are looking for

- **Crossover point, in mask-fraction terms, as a distribution.** The noise level below which Protocol C turns expansive and Protocol A distortion becomes large. Reported per-sample; if the distribution is bimodal (early-fork samples vs. smooth-drift samples), that is the finding — do not average it away.
- **Early-fork vs late-drift.** Free-running divergence dominated by a few early commit disagreements implies a schedule/threshold fix; smooth late-stage drift corroborated by Protocol C expansion implies genuine precision sensitivity near convergence.
- **Mitigation cost, priced honestly.** The implied mitigation — higher precision below the crossover mask fraction — requires keeping a second higher-precision weight copy resident or dequantizing on the fly. That cost is charged against the memory story in §8; a crossover at 40% mask fraction with a resident FP16 copy is not a ternary model.

4.4 Applicability caveat

Protocols A and B compare a quantized model against its FP16 twin, so they characterize **PTQ error dynamics**. Protocol C, notably, does not need a twin — it measures the FP16 dynamics' contractivity directly and applies unchanged to any model, including the from-scratch ternary line (run on the ternary model's own dynamics). For the QAT/from-scratch line (§5), twin-diff metrics are replaced by: Protocol C on the ternary model itself, self-consistency across seeds and restarts from the same canvas, oscillation rate, and — where a higher-bit QAT sibling is trained — KL against that sibling. Experiment 1′s twin results do not validate the shipping model; the two measure different objects.

This experiment is the core of Diffusal's science. Without it, the project is only generic quantization.

5. Experiment 2: Ternary Training as the Main Line

Objective: train the actual artifact — a natively ternary dLLM — and determine whether ternary hurts diffusion *more than it hurts autoregression*, which is the only question small-scale training can answer.

5.1 The end-goal build: Gemma 4 E2B, A2D + ternary QAT

The shipping artifact is an **A2D conversion of Gemma 4 E2B (~2.3B effective parameters) trained with ternary QAT, distilled from its FP16 parent**. The reasoning behind the pick:

- **Scale:** BitNet's competitiveness threshold is roughly 3B; E2B sits near it — the smallest Gemma 4 where ternary has a real chance, and the largest whose QAT fits a serious-hobbyist cloud budget. E4B (4.5B) is the stretch option if E2B results justify it; 12B/26B-A4B/31B are out of budget, and the 26B is MoE (router fragility, §6.3).
- **Recipe:** the dLLM/Tiny-A2D conversion recipe claims to convert any AR model; the FP16 parent doubles as both the distillation teacher and the matched FP16 AR control, making the gap-of-gaps measurement (§5) structurally free.
- **Pilot ladder:** the conversion+QAT pipeline is debugged on Gemma 3 270M and 1B (cheap, dense, vanilla architecture) before the E2B run. A pipeline bug discovered at E2B prices is a failure of process.
- **Known risk — Per-Layer Embeddings.** E2B reaches 2.3B *effective* parameters via PLE, so its embedding share is unusually large, and embeddings are excluded from ternarization by default (ablation grid below). The non-ternary fraction of the memory budget is therefore structurally higher than for a vanilla model; this is measured and charged against the §8 memory story, and if the PLE tables cannot be compressed safely, the E4B/dense-Gemma-3 fallback is reconsidered.
- **Double distribution shift, staged.** AR⟶diffusion conversion and FP16⟶ternary are each a distribution shift; doing both at once is the riskiest schedule. Default order: convert to diffusion at FP16 first, verify quality against the AR parent, then ternarize via QAT with the FP16 diffusion model as teacher. The one-stage variant is an ablation, not the plan.

The scale confound, stated up front. BitNet's own result is that ternary matches FP16 only from roughly 3B parameters upward; at the scales this program can afford, ternary QAT underperforms FP16 *for reasons unrelated to diffusion.* A small ternary dLLM losing to its FP16

twin is therefore the expected outcome under every hypothesis and discriminates nothing. The discriminating quantity is the **gap-of-gaps**:

> (ternary dLLM – FP16 dLLM) vs (ternary AR – FP16 AR), same scale, same tokenizer, same data, same token budget, same recipe.

If the dLLM's ternary gap is comparable to the AR ternary gap, diffusion adds no ternary fragility and the end-goal model is plausible at scale wherever ternary AR is. If the dLLM's gap is materially worse, refinement dynamics are implicated and §6′s fragility points are where to look.

At MDLM-owt scale, the training recipe and matched AR baseline already exist (§3.1), so this experiment reuses the MDLM codebase with ternary QAT added; the Tiny-A2D conversion recipe is the path for the 0.5B scale point and up to the §5.1 end-goal build, where the AR parent model doubles as the FP16 AR control at every rung.

Method: train small models under:

1. FP16 dLLM baseline,
2. native ternary QAT / from-scratch ternary dLLM,
3. FP16 AR control,
4. native ternary AR control (same recipe as 2),
5. PTQ variants from Experiment 0 as cheap negative baselines.

Ablation grid (dLLM line):

- all-ternary weights,
- all-ternary except embeddings/output head,
- all-ternary except mask embedding and remasking/confidence path,
- all-ternary except self-conditioning path if present,
- FP16 control.

Cost and variance plan. Under QAT, each ablation cell is a separate training run; the grid is 5 runs × ≥3 seeds for the dLLM line plus 2 × ≥3 seeds for the AR controls. Seed variance at small scale can swamp ablation deltas, so: report per-cell seed spread, and only claim an ablation effect when the between-cell difference exceeds the pre-registered multiple of within-cell spread. One-module-at-a-time ablations may be added for diagnosis but cannot replace the all-ternary configurations, and cells may be dropped for budget only from the bottom of this list, never the all-ternary or AR-control cells.

Stability measurement for the from-scratch model. Per §4.4: Protocol C run on the ternary model's own dynamics, seed/restart self-consistency from a fixed canvas, oscillation rate, and KL against a higher-bit QAT sibling where trained.

6. Special Fragility Points

6.1 Mask token and remasking schedule

The mask token is load-bearing in masked diffusion. Quantizing its embedding or the confidence/remasking logic may change which positions are refined or committed. Under commit-and-freeze sampling this is not one fragility among several — it is nearly the *entire* quantization question, since committed tokens are never revisited (§1). Under remasking samplers it remains the highest-leverage single path.

Default: keep the mask embedding and remasking/confidence computation higher precision until ablations prove they are safe.

6.2 Self-conditioning

If the model feeds previous-step distributions or denoising state back into the next step, that path may amplify quantization error.

Default: test self-conditioning separately and preserve it in FP16/INT8 if it causes trajectory divergence.

6.3 MoE routing

Routing errors are non-local: a small router perturbation can send tokens to different experts.

Default: routers stay FP16/INT8. Do not ternarize routers first.

7. Systems Reality Check

Ternary weights reduce storage, but current GPUs are optimized for INT4/INT8/FP16 tensor-core paths, not native ternary matmul. A ternary implementation may unpack into wider types and lose to INT4 despite using fewer bits on paper.

Expected systems result: ternary is slower than a good INT4 baseline until custom kernels prove otherwise.

If Experiment 1 recommends mixed precision below the crossover mask fraction, the serving design must carry either a second resident weight copy or on-the-fly dequantization; that cost belongs in every memory and latency number, not in a footnote.

Serving also should not start with vLLM. vLLM is optimized for autoregressive append loops and paged KV cache reuse. A diffusion model needs repeated full-canvas refinement. The lazy path is:

1. standalone fixed-canvas inference loop,
2. real latency/memory measurement,
3. custom kernels only if the loop is worth optimizing,
4. vLLM or other serving integration only after that.

8. Memory Reality Check

A hypothetical 25B ternary diffusion model is useful only as a warning, not a target. The arithmetic already argues against the old headline: **6 GB 25B deployment is probably not the Diffusal thesis.**

Approximate stress-case memory (qualitative, because the 25B target is fictional):

Component	Risk
Ternary weights	compact, but not free once scales/layout are counted
Higher-precision tail (post-crossover steps)	a resident FP16/INT8 copy can dominate the weight budget
Embeddings/output head	large vocab can cost gigabytes unless compressed
Activations	can dominate with canvas refinement and batching
Logits	large-vocab logits are expensive across refinement steps
Mask/remasking state	small in bytes, large in stability risk
Runtime buffers	allocator and kernel temporaries are not optional

The "6 GB is dead on arrival" claim must not rest on this qualitative table. It is only credible once we report a **measured peak-VRAM breakdown on the actual small model under test** — weights (including any mixed-precision tail), embeddings/head, per-step activations, logits, and runtime buffers as real numbers at the real canvas size, batch, and step count. That measurement is a required output of Experiments 0–1, not the fictional 25B row. If the measured activation-plus-logit fraction on the small model is already a large share of peak VRAM, the claim generalizes; if it is not, the claim is retracted.

The useful systems question is not "can a fictional 25B model fit in 6 GB?" It is:

> At a fixed VRAM budget, does a ternary dLLM beat the best smaller INT4/FP16 autoregressive baseline on quality, latency, and reliability?

If not, the end-goal model is a scientific result, not a deployment strategy — which is an acceptable outcome for this program, stated in advance.

9. Baselines and Go/No-Go Metrics

Every stage must compare against:

1. FP16 version of the same dLLM,
2. INT4 version of the same dLLM where available,
3. matched ternary/FP16 AR controls (§5) — same tokenizer, data, token budget, recipe,
4. smaller off-the-shelf autoregressive FP16/INT4 model at the same VRAM budget,
5. wall-clock latency on the target hardware.

Primary go/no-go: the gap-of-gaps (comparison 3). The kill decision rests on whether ternary hurts the dLLM more than it hurts the matched AR control, because that is the only comparison that isolates what Diffusal studies and is achievable confound-free at small scale.

AR frontier as context (comparison 4). At affordable scales, dLLMs currently lose to AR models on quality-per-GB *regardless of quantization*, so losing to the AR frontier does not kill the program — it fires on the diffusion-vs-AR gap, not on anything ternary. It is reported honestly as deployment context: if the end-goal model cannot beat the frontier even at scale, it ships as science, not as a serving strategy (§8). Where an exactly matched AR baseline cannot be trained within budget, the twin comparisons (1–2) are the fallback gate.

Secondary metrics:

- trajectory divergence by mask fraction (with quality anchor, per §4.2),
- Protocol C contraction rate by mask fraction,
- token recovery accuracy,
- oscillation rate (remasking sampler only),
- downstream task quality and oracle perplexity,
- constraint satisfaction where relevant,
- peak VRAM (including mixed-precision tail if used),
- tokens/sec or samples/sec at fixed quality.

All go/no-go thresholds are pre-registered before the relevant runs (§4.2).

10. Kill Criteria

Stop or re-scope when any condition is met. Each criterion references the pre-registered thresholds; divergence-only signals with no quality drop do not trigger kills (§4.2).

- **No diffusion-specific effect at Experiment 0:** the dLLM's PTQ degradation delta matches the AR control's across precisions — the thesis re-scopes from "refinement fragility" to "why is iterated denoising robust," and the build proceeds on the strength of the ternary-AR analogy alone.
- **Expansive dynamics with quality loss:** Protocol C shows expansion at noise levels covering most of the schedule, Protocol A distortion is large there, *and* the quality anchor drops past the declared threshold.
- **Mask-path fragility:** quantizing the mask embedding or remasking/confidence path changes commit decisions enough to miss quality thresholds.
- **Gap-of-gaps failure:** the ternary dLLM's gap to its FP16 twin exceeds the matched ternary AR gap by the pre-registered margin, and §6 mitigations (higher-precision mask path, self-conditioning, routers) do not close it.
- **Mitigation uneconomic:** the crossover sits at high mask fraction, so the required higher-precision tail erases the memory advantage (§4.3, §8).
- **No kernel win:** ternary inference is slower than INT4/FP16 after reasonable kernel effort.
- **Memory miss:** safe activation/embedding/logit compression still misses the target VRAM.

Losing to an *unmatched* off-the-shelf AR model is context, never a kill (§9).

11. Conclusion

Diffusal's end goal is a natively ternary text diffusion model in the Gemma family — an A2D-converted, ternary-QAT Gemma 4 E2B (§5.1). It should not be sold as "ternary makes huge diffusion LLMs fit on tiny GPUs" — the arithmetic does not support that. The sharper scientific thesis on the way to the build is error dynamics: **at which noise level does iterative denoising stop absorbing ternary quantization noise and start amplifying it, and is that behavior any worse than autoregression's at the same precision?**

The expected answer is a crossover distribution, not a verdict — absorption at high mask fraction, amplification near convergence — and the crossover is the result that matters because it prices the mitigation: a higher-precision tail that is cheap if the crossover is late and self-defeating if it is early. The mechanism to watch is argmax commitment acting as its own quantizer, absorbing sub-threshold error and discretizing the rest into irreversible commits. If diffusion's ternary gap matches autoregression's, the end-goal model inherits BitNet's scaling story and the build is justified. If it is worse, the negative result is still real: iterative refinement is more precision-sensitive than its weight count suggests. Either way the finding is only trustworthy if the trajectory comparison is well-posed (§4), the contractivity measurement is done on the dynamics rather than inferred from teacher-forced curves, and every quantization claim carries its matched autoregressive control.

References

- Ma et al., "The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits" (BitNet b1.58), arXiv:2402.17764.
- Sahoo et al., "Simple and Effective Masked Diffusion Language Models" (MDLM), arXiv:2406.07524.
- "Large Language Diffusion Models" (LLaDA), arXiv:2502.09992.
- Xu et al., "DLLMQuant: Quantizing Diffusion-based Large Language Models", arXiv:2508.14090.
- Christopher et al., "Constrained Discrete Diffusion", arXiv:2503.09790.
- Shang et al., "Post-training Quantization on Diffusion Models" (PTQ4DM), arXiv:2211.15736.
- Li et al., "Q-Diffusion: Quantizing Diffusion Models", arXiv:2302.04304.
- So et al., "Temporal Dynamic Quantization for Diffusion Models" (TDQ), arXiv:2306.02316.
- Wang, Schiff, Sahoo, Kuleshov, "Remasking Discrete Diffusion Models with Inference-Time Scaling" (ReMDM), arXiv:2503.00307. Code: github.com/kuleshov-group/remdm.
- MDLM checkpoints and matched AR/SEDD baselines: github.com/kuleshov-group/mdlm; huggingface.co/kuleshov-group/mdlm-owt.
- "dLLM: Simple Diffusion Language Modeling" (Tiny-A2D 0.5B/0.6B, AR→diffusion conversion recipes), arXiv:2602.22661. Code: github.com/ZHZisZZ/dllm.
- Tang et al., "Is Your Diffusion Sampler Actually Correct? A Sampler-Centric Evaluation of Discrete Diffusion Language Models", github.com/LuhanTang/dllm_sampler.
- DiffusionGemma-26B-A4B (Google DeepMind, open-weight block-diffusion MoE): huggingface.co/google/diffusiongemma-26B-A4B-it; ai.google.dev/gemma/docs/diffusiongemma.

- Gemma 4 (E2B/E4B/12B/26B-A4B/31B, Apache 2.0, April 2026): ai.google.dev/gemma/docs/core.